

Алгоритмы и структуры данных

Переводить человеческую задачу в последовательность шагов и отличать алгоритм от структуры данных.

DAY 06

ALGORITHM

complexity

data structure

stack

queue

ЦЕЛЬ ДНЯ

Что ты сможешь после этого дня

Конкретные действия вместо списка терминов.

- 01 Переводить человеческую задачу в последовательность шагов и отличать алгоритм от структуры данных.
- 02 Объяснить минимум 5 терминов темы своими словами.
- 03 Прочитать показанный пример кода и предсказать результат.
- 04 Самостоятельно решить похожую задачу после разбора.

СВЯЗЬ

Новая тема не появляется из ниоткуда

Связываем сегодняшний шаг с уже построенной картой курса.

ВЧЕРА

Как создают программы в команде

Что уже есть в нашей модели.



СЕГОДНЯ

Алгоритмы и структуры данных

Вчера: Как создают программы в команде. Сегодня мы добавляем следующий слой: алгоритмы и структуры данных.

ЗАЧЕМ

Зачем мне это?

Каждая тема решает конкретную проблему.

ПРОБЛЕМА → ПОЛЬЗА

Программирование начинается не с синтаксиса, а с умения разбить задачу на понятные действия и выбрать удобное представление данных.

Вопрос себе

Какую реальную проблему решает сегодняшняя тема?

Ответь до следующего слайда.

Сначала — человеческий образ

Потом обязательно связываем его с настоящей техникой.

БЫТОВОЙ ОБРАЗ

Алгоритм — рецепт приготовления, структура данных — форма, в которой лежат ингредиенты: список, очередь, стек или словарь.

ПЕРЕВОД В IT

algorithm

конечная последовательность шагов для решения задачи

complexity

оценка роста затрат времени или памяти

data structure

способ организации и хранения данных

Слой 1: algorithm • complexity • data structure

Сначала смысл, затем профессиональный термин.

**algorithm**

конечная последовательность шагов для решения задачи

**complexity**

оценка роста затрат времени или памяти

**data structure**

способ организации и хранения данных

Слой 2: stack • queue • hash table

Сначала смысл, затем профессиональный термин.

**stack**

структура, где последним забирают последний добавленный элемент

**queue**

структура, где первым забирают первый добавленный элемент

**hash table**

структура для быстрого доступа к значению по ключу

Слой 3: array • tree

Сначала смысл, затем профессиональный термин.

array

последовательная структура элементов

tree

иерархическая структура данных с узлами

Не заучивай формулировку. Придумай свой пример использования каждого термина.

Слой 4: graph • Big O

Сначала смысл, затем профессиональный термин.

**graph**

структура из вершин и связей между ними

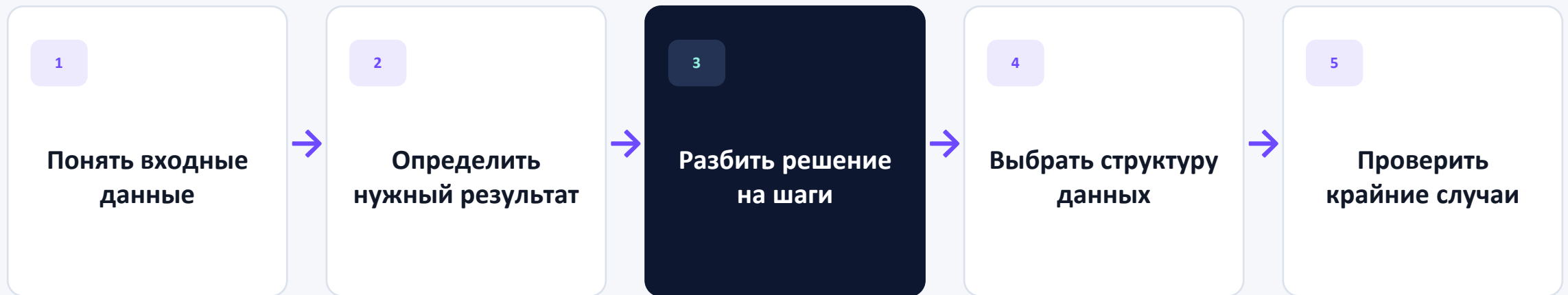
**Big O**

обозначение роста затрат алгоритма

Не заучивай формулировку. Придумай свой пример использования каждого термина.

Как это работает по шагам

Проследи движение слева направо.



Проговори эту цепочку своими словами, не подсматривая в текст.

Первый минимальный пример

Смотрим только на одну основную идею.



day06_step1

```
steps = ["взять заказ", "приготовить", "выдать"]  
  
for step in steps:  
    print(step)
```

ЧТО ИСКАТЬ

- Где данные?
- Где действие?
- Где результат?
- Что изменится, если заменить одно значение?

Разбираем пример построчно

Код не должен оставаться магической конструкцией.

```
steps = ["взять заказ", "приготовить",  
"выдать"]
```

Сохраняем значение или результат вычисления под понятным именем.

```
for step in steps:
```

Начинаем цикл и последовательно перебираем элементы.

```
print(step)
```

Показываем результат, чтобы увидеть состояние программы.

РЕЗУЛЬТАТ

Код → выполнение → результат

Всегда связываем запись с тем, что реально получилось.



до запуска

```
steps = ["взять заказ", "приготовить", "выдать"]  
  
for step in steps:  
    print(step)
```

РЕЗУЛЬТАТ

```
взять заказ  
приготовить  
выдать
```

ИЗМЕНИЛИ

Теперь изменим пример

Сначала предскажи результат. Не запускай код сразу.



changed

```
numbers = [3, 7, 2]
best = numbers[0]
for number in numbers:
    if number > best:
        best = number
print(best)
```



**Что появится в
результате?**

Сформулируй ответ словами.

ОТВЕТ

Сверяем прогноз

Не просто смотри ответ — объясни, почему он именно такой.

РЕЗУЛЬТАТ

7

ПОЧЕМУ

Python/система выполняет шаги в записанном порядке. Изменённые данные или действие дают новый результат.

Сравни с первым примером: что изменилось?

Как писать неправильно

Ошибка — отдельный учебный материал, а не повод её скрывать.

```
numbers = [10, 20, 30]  
print(numbers[3])
```

wrong

СТОП

**Что здесь может сломаться
или ввести нас в
заблуждение?**

Сначала найди проблему сам.

Почему возникает проблема

Переводим техническую причину на обычный язык.

У трёх элементов индексы 0, 1 и 2. Индекс 3 выходит за границы списка и вызывает IndexError.

Алгоритм отладки: симптом → конкретная строка/шаг → причина → минимальное исправление → повторная проверка.

Исправленный вариант

Меняем только то, что действительно было причиной.

```
numbers = [10, 20, 30]  
print(numbers[2])
```

fixed

ПРАВИЛО

Не переписывай всё решение. Найди минимальное осмысленное исправление.

Что изменилось после исправления?

Ответь без запуска.

- 1 Исчезла ли исходная причина проблемы?
- 2 Какой результат/поведение теперь ожидаем?
- 3 Как проверить это одним маленьким примером?

Сначала увидим полностью разобранный пример

Только после него будет самостоятельная практика.

ЗАДАЧА

Обработать очередь из трёх запросов строго в том порядке, в котором они пришли.

Не начинаем с кода. Сначала переводим задачу на человеческий язык.

КАК ДУМАТЬ

Как рассуждает программист

Разбиваем большую формулировку на маленькие действия.

- 1 **Храним запросы в списке.**
- 2 Берём элементы слева направо.
- 3 Для каждого выполняем одно действие.
- 4 Не перескакиваем через очередь.
- 5 После последнего запросов не остаётся.

Шаг 1: собираем основу

Не показываем 20 строк сразу — строим решение частями.

```
queue = ["req-1", "req-2", "req-3"]  
for request in queue:
```

solution_part1

Проверь

- Какие данные уже есть?
- Какой шаг плана реализован?
- Что ещё осталось добавить?

ИТОГОВЫЙ КОД

Шаг 2: итоговое решение

Теперь видим программу/запрос целиком.



solution_final

```
queue = ["req-1", "req-2", "req-3"]  
for request in queue:  
    print("обрабатываем", request)
```

Проверяем результат

Сравниваем то, что ожидали, с тем, что получили.

РЕЗУЛЬТАТ

обрабатываем req-1
обрабатываем req-2
обрабатываем req-3

ГЛАВНОЕ

Задача была решена не потому, что мы «знали готовый код», а потому что сначала построили понятный план.

ПОПРОБУЙ САМ

Практика 1

Задача очень похожа на разобранную — но теперь первый ход за тобой.

Составь алгоритм поиска максимального числа без функции `max()`.

Сначала выпиши 2–5 шагов решения словами.

ПОДСКАЗКА

Подсказка — только направление

Попробуй ещё раз до просмотра готового решения.

Нужна переменная с лучшим значением и сравнение каждого следующего элемента с текущим лучшим.

Вопрос себе: какие конструкции сегодняшнего дня нужны, а какие пока лишние?

Один из правильных вариантов

Сравни логику, а не количество строк.



practice_solution

```
numbers = [5, 2, 9, 4]
best = numbers[0]
for number in numbers[1:]:
    if number > best:
        best = number
print(best)
```

Ожидаемый результат

Если твой код устроен иначе, но даёт правильное и понятное решение — это нормально.

РЕЗУЛЬТАТ

9

Сравни

- Результат
- Понятность имён
- Лишние шаги
- Обработку ошибок

Практика 2 — мини-проект

Соединяем несколько элементов дня в одну маленькую закончённую задачу.

ЗАДАНИЕ

Сделай простую модель стека: добавить три действия и снять последнее.

Попробуй сначала без подсказки. Разрешено подсматривать в сегодняшние примеры.

Разбор мини-проекта

Вот один рабочий путь.



```
stack = []  
stack.append("open")  
stack.append("edit")  
stack.append("save")  
print(stack.pop())  
print(stack)
```

The image shows a dark-themed terminal window. In the top-left corner, there are three colored circles (red, yellow, green) representing a standard macOS window title bar. In the top-right corner, the text 'mini_project' is displayed. The main area of the terminal contains a Python code snippet that creates a list named 'stack', appends the strings 'open', 'edit', and 'save' to it, then prints the last element ('save') using 'pop()', and finally prints the entire list.

Что должно получиться

Связываем итог с задачей и проверяем смысл.

РЕЗУЛЬТАТ

```
save  
['open', 'edit']
```

Объясни каждую важную строку/команду своими словами. Если не можешь — вернись к конкретному месту, а не перечитывай весь день.

Четыре ловушки новичка

Лучше увидеть их здесь, чем случайно закрепить как привычку.

01

Начинать писать код до понимания задачи.

02

Выбирать структуру «по привычке», а не по операции.

03

Не проверять пустой ввод или один элемент.

04

Путать порядок очереди FIFO и стека LIFO.

Проверь себя: 1–2

Не листай дальше, пока не сформулируешь ответ.

1

Что такое «algorithm» и какую проблему это понятие помогает описать?

2

Чем «algorithm» отличается от «stack»? Объясни без копирования определения.

Проверь себя: 3–4

Здесь проверяем понимание кода и процесса мышления.

- 3 Какой результат даёт первый минимальный пример и почему?
- 4 Какую ошибку/риск мы разобрали сегодня и какое минимальное исправление нужно?

ОТВЕТЫ

Ответы к вопросам 1–2

Формулировка может отличаться — важен смысл.

1 **algorithm** — конечная последовательность шагов для решения задачи.

2 **algorithm**: конечная последовательность шагов для решения задачи. **stack**: структура, где последним забирают последний добавленный элемент.

ОТВЕТЫ

Ответы к вопросам 3–4

Сверяем причинно-следственную связь.

3

**взять заказ
приготовить
выдать**

4

У трёх элементов индексы 0, 1 и 2. Индекс 3 выходит за границы списка и вызывает `IndexError`.

ИТОГ

Сегодня мы научились

Не перечисляем слова — фиксируем новые способности.

- ✓ Переводить человеческую задачу в последовательность шагов и отличать алгоритм от структуры данных.
- ✓ Читать и объяснять пример темы «Алгоритмы и структуры данных».
- ✓ Находить типичную ошибку и формулировать причину.
- ✓ Решать похожую задачу по шагам, а не угадывать готовый код.

ДАЛЬШЕ

Что будет завтра

Связываем сегодняшний результат со следующим шагом.

Python и рабочее окружение

Завтра: Python и рабочее окружение. Сегодняшняя модель станет опорой для следующего шага.

Сегодняшняя тема не заканчивается здесь — она станет инструментом в следующем дне.